Percpu cycle

A **Per-CPU Variable** is a variable for which **each CPU has its own private copy**.

Instead of one shared variable:

int counter;

Linux creates:

CPU0 -> counter
CPU1 -> counter
CPU2 -> counter
CPU3 -> counter

Each CPU accesses only its own copy.

Why Per-CPU Variables?

Suppose all CPUs share:

int counter = 0;

and execute:

counter++;

Then:

- Race conditions can occur
- Synchronization (spinlocks/mutexes) is required
- Performance decreases

Using Per-CPU Variables:

DEFINE_PER_CPU(int, counter);

Each CPU updates its own counter.

No race condition occurs.

A CPU should **not access another CPU's copy**.

Per CPU Variables

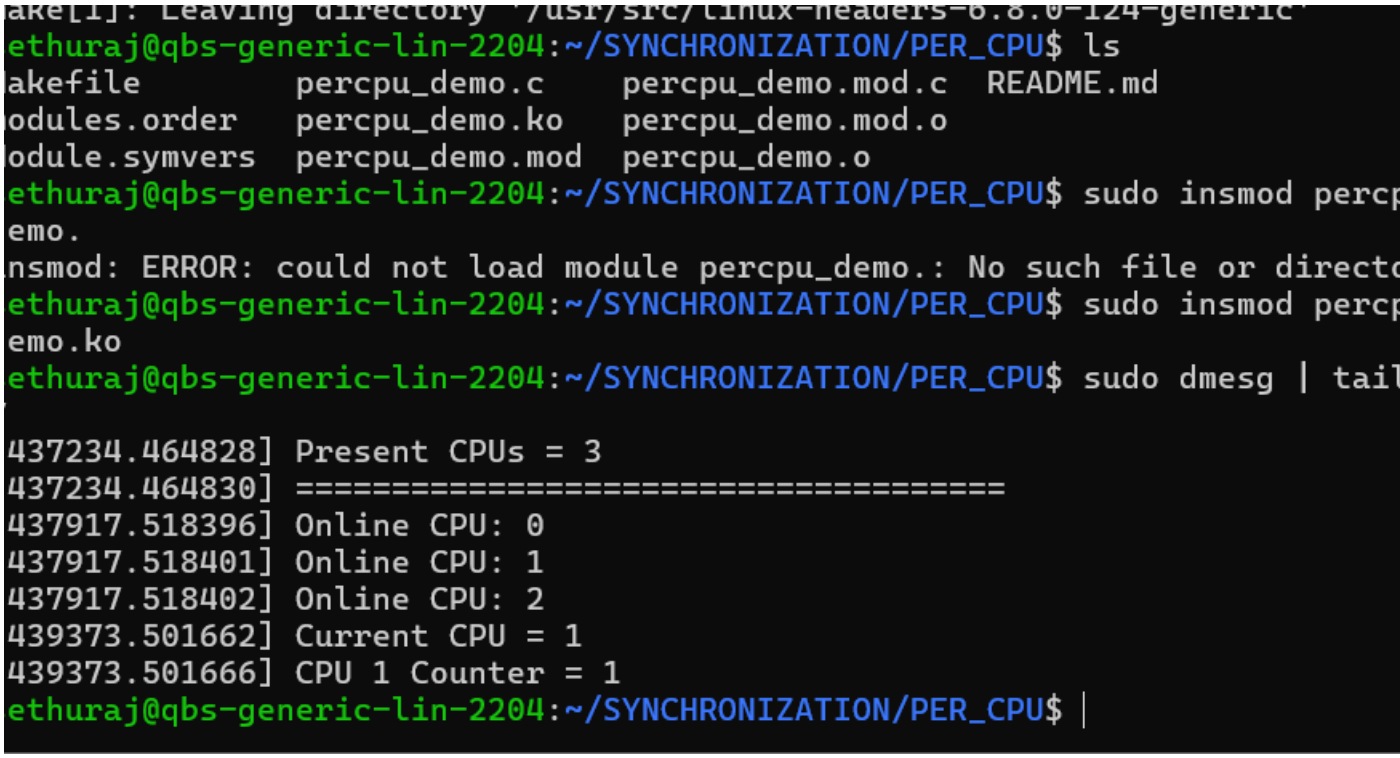
The simplest and most efficient synchronization technique consists of declaring kernel variables as per-CPU Variables.

Basically a per CPU Variables is an array of data structures, one element per each CPU in the system.

A CPU should not access the elements of the array corresponding to other CPU.

It can freely read and modify its own element without fear of race conditions, because it is the only CPU Entitled to do so.

The elements of the per-CPU array are aligned in main memory so that each data structure falls on a different line of the hardware cache



```
Skipping BTF generation for /home/sethuraj/SYNCHRONIZATION/PER_CPU_DS/per
_slides.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/PER_CPU_DS$ ls
Makefile         percpu_slides.c    percpu_slides.mod.c
modules.order    percpu_slides.ko   percpu_slides.mod.o
Module.symvers   percpu_slides.mod  percpu_slides.o
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/PER_CPU_DS$ sudo insmod p
pu_slides.ko
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/PER_CPU_DS$ echo 100 | su
tee /proc/percpu_entry
100
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/PER_CPU_DS$ cat /proc/per
_entry
--- Per-CPU Data Array State ---
percpu_data on cpu=0 is 0
percpu_data on cpu=1 is 100
percpu_data on cpu=2 is 0
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/PER_CPU_DS$ sudo dmesg |
l -10
[437234.464828] Present CPUs = 3
[437234.464830] ====================================
[437917.518396] Online CPU: 0
[437917.518401] Online CPU: 1
[437917.518402] Online CPU: 2
[439373.501662] Current CPU = 1
[439373.501666] CPU 1 Counter = 1
[439659.572493] Created /proc/percpu_entry
[439676.933808] proc_write_cpu called
[439676.933812] percpu_data on cpu=1 is 100
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/PER_CPU_DS$
```

```
get_cpu() on top of returning the current processor number also disables kernel preemption.
put_cpu() enables kernel preemption.

Why is disabling kernel preemption needed
=========================================

Just consider, for instance, what would happen
        if a kernel control path gets the address of its local copy of a per-CPU variable,
        then it is preempted and
        moved to another CPU: the address still refers to the element of the previous CPU.
```

1. Introduction

- Linux 2.6 introduced the **Per-CPU Interface (percpu)**.
- Used to create and manipulate Per-CPU variables.

- Simpler and more efficient than traditional Per-CPU arrays.
- Designed for large SMP (Symmetric Multiprocessing) systems.

2. Header File

#include <linux/percpu.h>

3. Advantages

- Avoids race conditions.
- Reduces locking overhead.
- Improves performance.
- Improves scalability on multicore systems.
- Each CPU accesses its own copy of data.

4. Creating Per-CPU Variables

Syntax

DEFINE_PER_CPU(type, name);

Example

DEFINE_PER_CPU(int, i);

- Creates one copy of i for every CPU.

Conceptually:

CPU0 → i
CPU1 → i
CPU2 → i
CPU3 → i

5. Creating Per-CPU Arrays

DEFINE_PER_CPU(int[3], my_array);

- Creates one array per CPU.

6. Declaring Per-CPU Variables

Syntax

DECLARE_PER_CPU(type, name);

Example

DECLARE_PER_CPU(int, i);

- Used when the variable is defined in another source file.
- Avoids compiler warnings.

7. Accessing Per-CPU Variables

get_cpu_var()

get_cpu_var(variable);

- Returns current CPU's copy of the variable.
- Disables kernel preemption.

put_cpu_var()

put_cpu_var(variable);

- Re-enables kernel preemption.
- Must be paired with get_cpu_var().

8. Example

DEFINE_PER_CPU(int, counter);

get_cpu_var(counter)++;

put_cpu_var(counter);

Steps:

1. Disable preemption
2. Access current CPU's counter
3. Increment counter
4. Enable preemption

9. Why Disable Preemption?

Without disabling preemption:

Read CPU0 variable
↓
Task migrates to CPU2

↓

Updates wrong CPU data

With:

get_cpu_var()

- CPU migration is prevented.

10. What is an lvalue?

Definition

- An lvalue (locator value) is an object that has a memory location.

Example:

int var;

var = 4;

- var is an lvalue.

11. Valid Example

var = 10;

- var has a memory address.

12. Invalid Examples

4 = var;

Error:

- 4 is not an lvalue.

(var + 1) = 4;

Error:

- (var + 1) is a temporary expression.

13. Why get_cpu_var() Returns an lvalue?

get_cpu_var(counter)++;

- Directly modifies the current CPU's copy.
- Returns the actual memory location of the Per-CPU variable.

Notes:

- Per-CPU Interface introduced in Linux 2.6.
- Header file: <linux/percpu.h>.
- Create variable: DEFINE_PER_CPU().
- Declare variable: DECLARE_PER_CPU().
- Access variable: get_cpu_var().
- Release variable: put_cpu_var().
- get_cpu_var() disables preemption.
- put_cpu_var() enables preemption.
- Improves SMP performance.
- Eliminates race conditions by giving each CPU its own data copy.

Per-CPU Data at Runtime – Point Wise Notes

1. What is Runtime Per-CPU Data?

- Sometimes the number or size of Per-CPU variables is not known at compile time.
- In such cases, Linux allows **dynamic allocation of Per-CPU data at runtime**.

2. Why Runtime Allocation?

- Size determined during execution.
- Dynamic memory requirements.
- Flexible allocation and deallocation.

3. Header File

```
#include <linux/percpu.h>
```

4. Allocating Per-CPU Data

Syntax

```
ptr = alloc_percpu(type);
```

Example

```
int __percpu *cpu_counter;

cpu_counter = alloc_percpu(int);
```

- Allocates one integer for each CPU.

Conceptually:

CPU0 -> int
CPU1 -> int
CPU2 -> int
CPU3 -> int

5. Accessing Current CPU's Data

*this_cpu_ptr(cpu_counter) = 100;

or

(*this_cpu_ptr(cpu_counter))++;

6. Accessing Specific CPU's Data

Syntax

per_cpu_ptr(ptr, cpu)

Example

*per_cpu_ptr(cpu_counter, 2) = 50;

Stores value in CPU2's copy.

7. Freeing Per-CPU Data

Syntax

free_percpu(ptr);

Example

free_percpu(cpu_counter);

Must be called before module removal.

9. Important APIs

API	Purpose
alloc_percpu(type)	Allocate Per-CPU memory
free_percpu(ptr)	Free Per-CPU memory
this_cpu_ptr(ptr)	Current CPU's pointer

API	Purpose
per_cpu_ptr(ptr, cpu)	Specific CPU's pointer

10. Compile-Time vs Runtime Per-CPU Data

Compile Time	Runtime
DEFINE_PER_CPU()	alloc_percpu()
Static allocation	Dynamic allocation
Fixed size	Flexible size
No explicit free	Must use free_percpu()